**BABEŞ-BOLYAI UNIVERSITY CLUJ-NAPOCA**
**FACULTY OF MATHEMATICS AND COMPUTER SCIENCE**
**SPECIALIZATION Artificial Intelligence**

# DIPLOMA THESIS

# Real-Time Intrusion Detection for Web Applications Using Deep Learning

**Supervisor**
**Alexandra–Ioana Albu**

*Author*
*Rares Marta*

2026

ABSTRACT

This thesis designs, implements, and evaluates a real-time intrusion detection system (IDS) for IoT networks using deep learning on the CICIoT2023 benchmark dataset, delivering a complete, reproducible pipeline from raw packet captures to live per-flow classification.

The system is built around a Multi-Layer Perceptron (MLP) trained at binary and 8-class (attack-family) granularities on a stratified random train/validation/test split, following common practice on CICIoT2023. The MLP is compared against a Random Forest baseline under identical conditions. On the binary task both models exceed the 0.85 weighted-F1 target, with the Random Forest reaching $\approx$0.96 and the MLP $\approx$0.93; on the harder 8-class task the Random Forest leads at $\approx$0.84 and the MLP attains $\approx$0.80, neither reaching 0.85, a shortfall traced to the limits of the public transport-layer features for separating application-layer attack families. Feature analysis identifies the per-flow packet count, the HTTPS/HTTP indicators, and header length as the most discriminative features, and prunes 14 of the 39 CICIoT2023 features (near-zero-variance flags and redundant continuous columns) to leave a 25-feature model. Temperature scaling calibrates the MLP's softmax confidences.

The practical component is a web demonstration, NEURAL IDS, pairing a FastAPI backend with a React dashboard that accepts pre-extracted feature CSVs or raw packet captures, classifies flows in under 10 milliseconds, and displays confidence scores and per-class probability breakdowns. It correctly flags a synthetic SYN-flood capture as an attack (every flow flagged as Attack, mean calibrated confidence 99.6%) and classifies benign browsing as Benign (98% of flows), validating the core classification functionality required by the project specification.

# Contents

# Chapter 1

# Introduction

## 1.1 Context and Motivation

Intrusion Detection Systems (IDS) are a core defensive layer in modern networks, monitoring traffic to detect malicious behaviour such as denial-of-service attacks, reconnaissance scans, brute-force attempts, and data exfiltration. The classical approach is signature-based detection, in which traffic is matched against a database of hand-written rules describing known attacks (the approach behind tools such as Snort [22]). Signature matching is fast and precise on threats it already knows, but it has two structural weaknesses: it cannot recognise an attack for which no rule has been written, and the rule set must be maintained by hand as new threats appear.

These weaknesses have become more pressing with the growth of the Internet of Things (IoT). Modern deployments place large numbers of constrained, infrequently patched devices on the same networks as critical services, and the volume and diversity of the resulting traffic strain a detection model that depends on enumerating every attack in advance. The same devices are attractive targets: compromised IoT endpoints are routinely conscripted into large botnets and used to launch volumetric denial-of-service campaigns, which is reflected in benchmark data where denial-of-service and distributed denial-of-service flows dominate the attack distribution [21].

Machine learning, and deep learning in particular, offers an alternative that learns the statistical structure of traffic directly from data rather than from hand-written rules [9, 17]. A model trained on labelled flows can, in principle, generalise to variants of known attacks and flag anomalous behaviour without an exact signature. This premise has produced a large literature reporting strong benchmark accuracy for neural intrusion detection, including work that applies deep models to real-time web intrusion detection rather than offline analysis alone [3, 28].

## 1.2 The Research-to-Deployment Gap

A model that scores well on a benchmark dataset is not automatically a model that works in practice [5]. This thesis pays attention to three practical problems that are easy to overlook when only the benchmark score matters.

The first problem is how the data is split for evaluation. If the split is random, flows recorded seconds apart in the same capture session can end up on both sides of the train/test boundary. The model is then tested on traffic it has, in effect, already seen, and the measured accuracy is higher than what it would achieve on genuinely new traffic. The second problem is time. Network traffic changes as applications, user habits, and attack tools evolve [4], and a deployed model always works on traffic that is newer than its training data. An evaluation that ignores this order says little about how the model will behave once deployed. The third problem is trust in the reported confidence. A detector does not only output a label; it shows a confidence score next to it, and neural networks tend to report confidences that are higher than their actual accuracy unless those scores are explicitly calibrated [12]. On top of these three, a real-time system has a latency budget: classifying a flow has to be fast enough to be useful, something an offline experiment never has to prove.

These concerns shape the design of this work. The evaluation uses a stratified random split, following common practice on CICIoT2023; the leakage and temporal-ordering concerns raised above are revisited as limitations and future work. The neural model is compared against a strong tree-based baseline under identical conditions. The confidence scores are calibrated before being shown to the user, and inference latency is measured against an explicit acceptance criterion.

## 1.3 Approach and Scope

The work is built on the CICIoT2023 dataset, a large-scale IoT intrusion-detection benchmark of over 46 million labelled flow records spanning 33 attack types, captured in a 105-device smart-home testbed [21]. The primary classifier is a Multi-Layer Perceptron (MLP) trained at two granularities: a binary attack-versus-benign task and an eight-class task over coarse attack families. Because the public CICIoT2023 features are tabular flow statistics rather than raw packets, the deep model is not assumed to be the best tool by default; recent evidence shows that well-tuned simple neural networks are competitive with gradient-boosted trees on tabular data [14], and the thesis tests this empirically by comparing the MLP against a Random Forest baseline on the same split. Beyond offline evaluation, the trained model is deployed behind a live demonstration in which scripted attacks are launched against a controlled target and the resulting flows are classified in near real time.

The scope is bounded in two respects. The system is trained and evaluated only on the 39-feature public release of CICIoT2023, which is derived from packet headers and flow metadata and carries no payload features; detection of application-layer attacks, whose signal lies in the payload, is therefore expected to be limited, and the results chapter reports this rather than working around it. Generalisation beyond the smart-home setting of the dataset is not claimed.

## 1.4 Objectives and Contributions

The thesis makes six contributions: (1) a reproducible end-to-end pipeline from the raw CICIoT2023 captures to a live per-flow classifier, with deterministic seeding and a fixed artefact contract; (2) an evaluation of an MLP against a Random Forest baseline at binary and eight-class granularities; (3) a feature analysis that prunes the 39 public features to a 25-feature set by dropping near-zero-variance flags and redundant continuous columns; (4) confidence calibration of the deep model by temperature scaling [12], so the confidence shown to a user reflects empirical accuracy; (5) a deployed demonstration system, NEURAL IDS, comprising a containerised inference backend and a web dashboard, validated against both benign and synthetic-attack traffic; and (6) an account of where the approach falls short, particularly the eight-class attack-family task, where the transport-layer-only public features cap performance below the 0.85 weighted-F1 bar that the binary task meets.

## 1.5 Thesis Structure

The remainder of this thesis is organised as follows. Chapter 2 presents the theoretical background, covering intrusion detection systems, deep learning for traffic classification, and the CICIoT2023 dataset. Chapter 3 presents the machine learning methodology: the data pipeline, the split strategies, the model architecture and training procedure, and the evaluation protocol. Chapter 4 reports the experimental results of that methodology and discusses the findings. Chapter 5 then states the requirements and specification of the application built around the trained model, and Chapter 6 describes its design, implementation, and testing, including the verification of the core functionality. Chapter 7 concludes the thesis and outlines directions for future work.

## 1.6 Declaration on the Use of Generative AI Tools

Generative AI tools were used during the preparation of this thesis: AI coding assistants supported the development of the training pipeline and the demonstration application, and AI language models helped draft and revise portions of the text. All such output was reviewed, tested, and edited by the author, and every experimental result was produced by running the pipeline and verified independently of any AI tool. The author takes full responsibility for the content of the thesis, the correctness of the results, and the final wording.

# Chapter 2

# Theoretical Background

This chapter is organised in two parts. Section 2.1 sets out the theoretical notions the rest of the thesis depends on: what network traffic is and how it is monitored, what an intrusion detection system does, the multi-layer perceptron used as the primary model, and the CICIoT2023 dataset used for training and evaluation. Section 2.2 then reviews the machine learning and deep learning methods applied to intrusion detection and the recurring challenges reported across that literature.

## 2.1 Theoretical Notions

### 2.1.1 Network Traffic and Monitoring

Network traffic consists of data packets transmitted between devices across a network. Data does not travel as one solid block: when a client issues an HTTP request, the request is broken into packets as it moves down through the network layers, each packet carrying header information (addresses, ports, protocol flags) and a fragment of the payload, routed independently and reassembled at the destination [7]. By direction, traffic is commonly described as north–south (between clients and servers, crossing the network boundary) or east–west (between servers inside the network). The header and timing statistics of these packets, their sizes, rates, flags, and inter-arrival times, are the raw material from which the flow features used in this thesis are derived.

Network monitoring is the continuous observation and analysis of a network's traffic and performance. Beyond its operational goals, it underpins cybersecurity, the practice of protecting systems, networks, applications, and data from digital attacks and unauthorised access [11]: monitored traffic is examined for patterns, anomalies, and potential security threats. Artificial intelligence is reshaping this field on both sides, with attackers using it to scale phishing, generate deepfakes, and automate exploits, and defenders using it to improve detection and accelerate response. The

intrusion detection systems discussed next are the monitoring components dedicated to that defensive role.

### 2.1.2 Intrusion Detection Systems

An Intrusion Detection System (IDS) is a security mechanism that monitors network traffic or host activity to identify malicious behavior. As networks have grown in scale and complexity, traditional perimeter defenses such as firewalls have proven insufficient on their own: attackers routinely bypass static rules and exploit new vulnerabilities, while modern networks exhibit high throughput, heterogeneity, and continual change [17]. IDS complement these defenses by analyzing traffic patterns and flagging suspicious activity.

IDS can be broadly categorized into two paradigms based on their detection methodology: signature-based and anomaly-based systems.

#### Signature-Based Detection

Signature-based IDS, exemplified by tools such as Snort and Suricata, operate by scanning network traffic for predefined patterns of known malicious activity [22]. These signatures can describe byte sequences, header anomalies, or protocol violations. Signature-based systems are highly effective at detecting known attacks with high precision, and they produce few false positives when the signature database is well-maintained.

However, their static nature makes them fundamentally limited against novel threats. Zero-day exploits, polymorphic attacks, and previously unseen attack variants will not match any existing signature and will therefore pass undetected. Signature databases also require continuous manual updates to remain effective, which introduces an inherent lag between the emergence of a new attack and the availability of its corresponding signature.

#### Anomaly-Based Detection

Anomaly-based IDS take a different approach: instead of matching against known attack patterns, they learn a model of normal network behavior and flag deviations from this baseline as suspicious. This paradigm has the significant advantage of being able to detect previously unseen attacks, since any traffic that deviates sufficiently from the learned normal profile will trigger an alert.

The trade-off is a higher false positive rate. Unusual but legitimate traffic (such as a sudden spike in usage during a product launch, or an uncommon protocol used by a newly deployed application) can be incorrectly flagged as malicious. The sensitivity

of anomaly-based systems depends heavily on the quality of the normal behavior model and on the threshold chosen to separate normal from anomalous activity.

Figure 2.1 illustrates the typical architecture of a real-time deep learning-based IDS, showing the end-to-end pipeline from raw packet capture to alert generation.

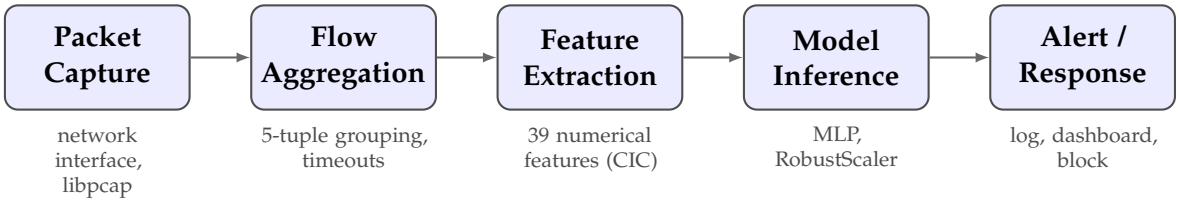


Figure 2.1: End-to-end architecture of a real-time deep learning-based intrusion detection system.

From a machine learning perspective, intrusion detection maps naturally to classification. In its simplest form, the task is binary: distinguish benign traffic from malicious flows. Multi-class setups are also common, with systems trained to recognize distinct categories of attacks such as DDoS, denial-of-service, reconnaissance, or botnet activity. The input is typically tabular flow data, where each flow is described by numerical features such as packet counts, byte rates, and TCP flag statistics [23].

#### Detection versus Prevention: IDS and IPS

Intrusion *detection* and intrusion *prevention* systems share the same analysis techniques but differ in placement and authority. An IDS observes a copy of the traffic from beside the path and raises alerts, leaving the response to an operator or another system. An Intrusion Prevention System (IPS) sits inline, in the path itself, and can drop malicious packets before they reach their target [10]. Inline authority comes at a price: a false positive now blocks legitimate traffic, and the IPS adds latency and a point of failure to every packet it forwards. A common middle ground is to detect beside the path and respond by installing a firewall rule against the offending source, which stops the remainder of an attack without taking inline risk.

Any response local to the protected host also runs into a physical limit: bandwidth is consumed before the host has any say. Dropping a packet at the local firewall is free, but the bandwidth that packet spent travelling the wire is already gone. If a botnet directs tens of gigabits per second at a server on a one-gigabit link, even a perfect and instantaneous blocking rule cannot help: the flood saturates the link upstream of the host's authority, and legitimate packets are lost before the kernel ever sees them. Volumetric attacks are therefore ultimately mitigated upstream, by the network provider or by cloud-based mitigation services [8], while host-level detection and response remain effective against attacks that fit within the link, such as scans, brute-force attempts, and application-layer abuse.

### 2.1.3 The Multi-Layer Perceptron

A Multi-Layer Perceptron (MLP), also referred to as a fully connected feedforward neural network or deep neural network (DNN), is the foundational deep learning architecture for supervised classification. An MLP consists of an input layer, one or more hidden layers, and an output layer, where each layer is fully connected to the next.

Formally, given an input feature vector $\mathbf{x} \in \mathbb{R}^d$ representing a network flow described by $d$ numerical features, each hidden layer $l$ computes:

$$\mathbf{h}_l = \sigma(\mathbf{W}_l \mathbf{h}_{l-1} + \mathbf{b}_l) \tag{2.1}$$

where $\mathbf{W}_l$ and $\mathbf{b}_l$ are the learnable weight matrix and bias vector of layer $l$, and $\sigma$ denotes a non-linear activation function, typically the Rectified Linear Unit (ReLU), defined as $\text{ReLU}(x) = \max(0, x)$. The input to the first hidden layer is the feature vector itself: $\mathbf{h}_0 = \mathbf{x}$.

For binary classification (benign vs. attack), the output layer applies a sigmoid activation:

$$\hat{y} = \sigma_{\text{sig}}(\mathbf{w}^T \mathbf{h}_L + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{h}_L + b)}} \tag{2.2}$$

where $\mathbf{h}_L$ is the output of the last hidden layer and $\hat{y} \in (0, 1)$ represents the predicted probability that the input flow is an attack. The model is trained by minimizing the binary cross-entropy loss:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \tag{2.3}$$

where the loss is averaged over the $N$ training samples and minimised by gradient descent with backpropagation, typically using the Adam optimizer [16]. The multi-class generalisation, used for the attack-family task in this thesis, replaces the sigmoid output with a softmax over classes and the binary loss with the categorical cross-entropy of Equation (3.4) in Chapter 3.

### 2.1.4 The CICIoT2023 Dataset

The CICIoT2023 dataset, published by the Canadian Institute for Cybersecurity (CIC) at the University of New Brunswick, is a large-scale benchmark dataset for IoT intrusion detection research [21]. It was selected for this thesis based on several criteria: recency (2023), scale (over 46 million records), diversity of attack types (33 distinct attacks across 7 categories), and growing adoption in the research community.

### Testbed and Data Collection

The dataset was generated in a physical laboratory simulating a realistic smart-home network of **105 real IoT devices** (smart cameras, speakers, plugs, lights, sensors, and home-automation hubs), of which 67 are directly involved in attack scenarios as attackers or victims. Attacks are launched from 7 Raspberry Pi devices, coordinated over SSH for the distributed scenarios, and traffic is captured passively at a network tap, producing approximately 548 GB of raw PCAP data.

Feature extraction is performed using **DPKT**, a Python packet parsing library. Unlike CICFlowMeter (used in earlier CIC datasets such as CICIDS2017), DPKT operates at the packet level with a fixed-size packet window, computing features over a sequence of consecutive packets between two hosts rather than per-flow. This approach offers finer granularity and is more suitable for real-time detection, as features can be computed as soon as enough packets in the window are observed, without waiting for a flow to terminate [21].

### Attack Taxonomy

The dataset contains **33 distinct attack types** organized into 7 categories, plus benign traffic, for a total of 34 class labels. Table 2.1 summarizes the attack categories, the number of attacks in each, and the tools used for generation.

| Category | Count | Examples | Tools |
|---|---|---|---|
| DDoS | 12 | UDP Flood, SYN Flood, SlowLoris, HTTP Flood | hping3, golang-httpflood |
| DoS | 4 | TCP Flood, UDP Flood, SYN Flood, HTTP Flood | hping3, udp-flood |
| Reconnaissance | 5 | PingSweep, PortScan, OSScan, VulnerabilityScan | nmap, fping, vulscan |
| Web-Based | 6 | SQL Injection, XSS, Command Injection, Backdoor | DVWA, Remot3d, BeEF |
| Brute Force | 1 | DictionaryBruteForce | nmap, hydra |
| Spoofing | 2 | ARP Spoofing, DNS Spoofing | ettercap |
| Mirai | 3 | GREIP Flood, GREeth Flood, UDP-Plain | Adapted Mirai source |

Table 2.1: CICIoT2023 attack taxonomy: 7 categories with 33 attack types.

### Feature Set

The publicly available CSV release distributed by the Canadian Institute for Cybersecurity provides 39 features per flow, used for all experiments and the live demonstration in this thesis. The features cover flow timing (duration, time-to-live, inter-arrival time), packet rates, header and protocol information, binary TCP-flag indicators and flag counts, binary protocol indicators (HTTP, HTTPS, DNS, TCP, UDP, and others), packet-length statistics, and aggregate statistics computed over the combined incoming and outgoing packets of a window.

This is the only feature representation the official CIC distribution provides. Table 2.2 summarises the 39 features grouped by family.

| **Family** | **Features (39; *14 dropped*)** | **Description** |
|---|---|---|
| Flow metadata | Header_Length, Protocol Type, Time_To_Live | Header byte length, transport protocol id, IP time-to-live. |
| Rate & timing | Rate, IAT | Packet transmission rate; mean inter-arrival time. |
| Packet-size statistics | Tot sum, Min, Max, AVG, Std, Number, *Tot size*, *Variance* | Aggregate packet-length statistics over the flow and the packet count (*Tot size, Variance* dropped as redundant). |
| TCP flag values (binary) | FIN, SYN, RST, PSH, ACK, *ECE*, *CWR* | 1 if the flag is set on any packet in the flow (*ECE, CWR* dropped: near-zero variance). |
| TCP flag counts | ACK, SYN, FIN, RST | Number of packets in the flow with that flag set. |
| Protocol indicators (binary) | HTTP, HTTPS, DNS, TCP, UDP, *Telnet*, *SMTP*, *SSH*, *IRC*, *DHCP*, *ARP*, *ICMP*, *IGMP*, *IPv*, *LLC* | 1 if the flow uses that protocol (10 dropped: near-zero variance, almost always 0). |

Table 2.2: CICIoT2023 flow features (39-feature public CSV), grouped by family. The 14 italicised features are dropped by feature selection (near-zero variance or redundant), leaving the 25 used for all experiments.

### Dataset Statistics and Class Distribution

The full dataset contains approximately 46.7 million records across 169 CSV files, about 13 GB uncompressed, and its class distribution is severely imbalanced. Benign traffic accounts for roughly 1.1 million samples (2.4% of the dataset); DDoS attacks make up about 73% of all records, with DDoS-ICMP_Flood alone contributing over 7.2 million samples; and the web-based attacks together amount to fewer than 25,000 samples, with the rarest classes (XSS, Backdoor_Malware, CommandInjection) holding only around 100 samples each.

The most extreme class ratio is approximately 5,751:1 between DDoS-ICMP_Flood and the smallest attack class. This imbalance must be addressed during training, whether by class weighting, stratified sampling, or synthetic oversampling such as

SMOTE [6]. Due to the dataset's size and computational constraints, most published experiments operate on stratified subsamples of 1–5 million records, preserving the original class distribution [21].

**Comparison with Earlier Datasets**

Table 2.3 compares CICIoT2023 with other commonly used IDS benchmark datasets.

| Dataset | Year | Records | Features | Attack types | Domain |
|---|---|---|---|---|---|
| NSL-KDD [25] | 2009 | 150K | 41 (custom) | 4 classes | General |
| UNSW-NB15 [20] | 2015 | 2.5M | 49 (Argus+Bro) | 9 classes | General |
| CICIDS2017 [23] | 2017 | 2.8M | 78 (CICFlowMeter) | 14 attacks | Enterprise |
| CICIoT2023 [21] | 2023 | 46.7M | 39 (DPKT, CSV) | 33 attacks | IoT smart home |

Table 2.3: Comparison of commonly used IDS benchmark datasets.

CICIoT2023 offers several advantages over older benchmarks: it is significantly larger, contains more diverse and modern attack types (including IoT-specific Mirai variants), and uses real IoT hardware rather than simulated traffic generators. Its limitations should equally be acknowledged [21]. Attacks are executed individually in a controlled laboratory, not as the multi-stage, adaptive campaigns of real attackers. The features come from packet headers and metadata only, with no encrypted-payload inspection, which matters as IoT traffic becomes increasingly encrypted. The single smart-home domain limits transfer to industrial or healthcare IoT, the class imbalance is severe (Section 2.1.4), and the exact formulas of some derived features (Magnitude, Radius, Covariance, Variance, Weight) are not fully documented in the original paper.

## 2.2 Literature Review

### 2.2.1 Machine Learning and Deep Learning for Intrusion Detection

Traditional machine learning approaches to IDS (such as Support Vector Machines, Decision Trees, and Random Forests) depend on manually engineered features and often struggle with high-dimensional, imbalanced, and non-stationary data. Deep learning offers an alternative by learning hierarchical representations directly from raw or lightly processed inputs. In the IDS domain, the literature reports many

deep learning architectures, including deep feed-forward networks, convolutional neural networks (CNNs), recurrent neural networks (RNNs) with Long Short-Term Memory (LSTM) units, autoencoders, and more recently transformer-based models with self-attention [17].

### The Multi-Layer Perceptron in Intrusion Detection

MLPs (Section 2.1.3) are well suited to flow-level intrusion detection. Network flow features form fixed-length numerical vectors, exactly the input format MLPs expect, and unlike CNNs (which assume spatial structure) or RNNs (which assume sequential structure) an MLP makes no structural assumption about the input, which fits independent flow records with heterogeneous features [14]. Inference is a sequence of matrix multiplications that modern hardware executes quickly, so a small MLP can process tens of thousands of flows per second on a single CPU core, which matters for low-latency detection [26]. Well-regularised MLPs are also competitive on tabular data: Kadra et al. showed that an MLP with a tuned combination of dropout, weight decay, and batch normalisation outperformed both XGBoost and specialised architectures such as TabNet on nearly half of the 40 datasets they evaluated [14]. In IDS research MLPs routinely reach 95–99% accuracy on standard benchmarks, which makes them a competitive baseline against which to measure more complex architectures [26, 9].

Training an MLP for IDS turns on a few standard choices. Class imbalance is handled by weighting the minority (attack) classes more heavily in the cross-entropy loss, or by oversampling methods such as SMOTE [6], or by focal loss that down-weights easy examples. Regularisation combines dropout (zeroing neuron outputs during training with probability typically between 0.2 and 0.5) [24], weight decay, and early stopping on the validation loss. Feature scaling is essential because flow features span very different ranges (packet counts versus flow durations in microseconds), so a scaler is fitted on the training set and applied unchanged to the validation and test sets.

### Other Deep Learning Architectures for IDS

Several other architectures have been applied to intrusion detection. Convolutional networks reshape tabular features into one- or two-dimensional grids and use convolutions to extract local patterns tied to protocol-specific or timing signatures [27]. Recurrent networks with LSTM units capture temporal dependencies in sequential traffic, which helps in industrial or cyber-physical settings where context over time matters [15]. Autoencoders learn to reconstruct their input through a bottleneck and, when trained only on benign traffic, use the reconstruction error as an anomaly

score; this needs no labelled attacks but depends heavily on the chosen threshold and on training-data quality [19]. Transformers use self-attention to model global relationships among features, treating each feature as a token, but they are usually supervised and more compute-intensive than the alternatives [28]. Compared with these, the supervised MLP used in this thesis needs labelled data and is less sensitive to genuinely novel attacks, but it trains and serves quickly and calibrates more easily than threshold-based anomaly detectors.

### 2.2.2 Explainability and Feature Attribution

A recurring objection to deep learning for intrusion detection is opacity: a network outputs a verdict but no reason for it, and an alert that cannot be justified is hard to act on or to audit. This has made explainable AI (XAI) a distinct strand of the IDS literature, where post-hoc attribution methods expose which inputs a trained detector relied on, turning a black-box score into something an operator can inspect and contest [2]. The need is sharper in security than in many other settings: a false alert carries an investigation cost and a missed one a breach, so the feature that drove a decision is itself operationally useful information.

Attribution methods divide into two scopes. *Global* attribution asks which features matter to the model across the whole dataset; permutation importance, which measures the drop in a metric when one feature's values are shuffled to break its association with the label, is a standard model-agnostic example, and it is the method used in Section 4.3 to audit and justify the pruned feature set. *Local* attribution instead explains a single prediction, assigning each feature a signed contribution to that one verdict. The most widely used local method is SHAP (SHapley Additive exPlanations), which borrows the Shapley value from cooperative game theory: it treats the features as players and distributes the gap between the model's output and a baseline expectation among them as additive contributions, with the guarantee that the attributions sum exactly to that gap [18]. A global ranking says which features the detector trusts in general; a local SHAP explanation says which features made *this* flow look like an attack.

Both scopes appear in this thesis. The offline feature analysis (Section 4.3) uses global permutation importance to confirm that the 25 retained features carry the discriminative signal and that the pruned columns do not. The demonstration system (Chapter 6) takes the local view: each flagged flow can be accompanied by an on-demand SHAP attribution showing which features pushed its verdict, so the operator sees not only that a flow was called an attack and with what confidence, but on what grounds. The explanation is computed on request rather than for every flow, because per-prediction Shapley estimation is markedly more expensive than the forward pass

it explains, a cost reflected in the demo's reported processing time (Figure 6.2).

### 2.2.3 Challenges in Machine-Learning-Based Intrusion Detection

Deep learning models for IDS operate under several challenges that frequently undermine the transfer from benchmark performance to deployment. Class imbalance is the most visible: benign traffic usually outnumbers attack traffic, and benchmark datasets reflect this. In CICIoT2023 DDoS attacks account for roughly 73% of all records while web-based attacks represent less than 0.1% [21], which can push a model to favour the majority class and makes accuracy a misleading metric unless weighted loss functions, stratified sampling, or oversampling such as SMOTE [6] are used. Concept drift compounds the problem: traffic distributions change over time as user behaviour, applications, protocols, and attacker strategies evolve, so a model trained on a static dataset may degrade once the traffic it sees diverges from its training distribution [4]. A third difficulty is dataset realism, since most benchmarks are generated in controlled testbeds rather than captured from production networks; testbeds allow precise labelling but may not capture encrypted communications, multi-stage attacks, or the noise of large-scale networks [17]. Finally, deployment constraints, latency and throughput budgets, integration with existing infrastructure, and long-term maintainability are underreported in a literature focused on accuracy on static benchmarks [5].

# Chapter 3

# Machine Learning Methodology

This chapter presents the machine learning core of the thesis: how the raw CIC-IoT2023 captures are turned into clean training data, how that data is split so that the evaluation is trustworthy, how the model is built and trained, and how it is evaluated. Everything else in the system is built on top of the artefacts this chapter produces. The application that serves the trained model is specified in Chapter 5 and designed in Chapter 6; the experimental results of the methodology described here are reported in Chapter 4.

## 3.1 Classification Tasks

The dataset provides 34 fine-grained labels (the 33 attack types of Section 2.1.4 plus benign traffic). This thesis does not classify at that finest level. Instead, the same rows are relabelled into two classification tasks of practical interest:

- **Binary (2-class)**: benign versus attack. This is the task an alerting IDS uses to decide whether traffic is malicious at all, and it is the primary task of the thesis.
- **8-class**: the seven attack families of Table 2.1 (DDoS, DoS, Reconnaissance, Web-based, Brute Force, Spoofing, Mirai) plus benign. This is the coarser attack-family view an analyst consumes, reported as a secondary task.

Both tasks are derived from the same subsampled rows and the same train/validation/test partitions; only the label mapping changes. The 34-class fine-grained task is not modelled, because the rarest variants hold too few unique records (Section 2.1.4) to train and evaluate reliably, and because attack-family granularity is sufficient for the alerting and triage use cases the demonstration targets.

## 3.2 Data Pipeline

The training data pipeline performs a sequence of deterministic transformations from the raw per-folder CSVs to the train/validation/test tensors that feed the model. Each transformation is a separate pipeline phase so that intermediate state is inspectable.

### 3.2.1 Ingestion and Deduplication

For each of the attack folders, the pipeline lazily scans every CSV, projects only the feature columns plus a synthetic column that carries the source-capture filename, and concatenates the resulting lazy frames. Materialisation happens only after the projection, so the multi-tens-of-millions-of-rows corpus is never held in memory in its raw width.

CICIoT2023 contains a substantial fraction of exact duplicate rows, a known property of the dataset and a recurring concern in the literature. Deduplication is applied immediately after concatenation, before any sampling. Deduplicating before sampling is essential: a folder where most rows are duplicated would otherwise have the same row repeated many times in the sample, which would inflate accuracy on the corresponding test rows. The quantitative impact of each pipeline stage (raw count, duplicates removed, unique rows, sampling cap applied) is reported in Chapter 4 (Figure 4.1).

### 3.2.2 Per-Class Subsampling

The *per-class cap* is a fixed upper bound on the number of rows kept for each fine-grained class, set to 200,000 rows in this work. After deduplication, every folder whose deduplicated row count exceeds this cap is randomly subsampled down to it with a fixed seed, while folders below the cap are kept in full. This brings the effective training-set size down from tens of millions of rows to a few million while keeping every attack class represented.

The cap is set at the high end of the range used in the CICIoT2023 literature. Random sampling is preferred over taking the leading rows of each folder because each attack folder is a chronological concatenation of several capture sessions: the first $N$ rows come almost entirely from the first session, and a leading-rows sample would bias the model toward whatever happened in that opening session.

### 3.2.3 Caching

The deduplicated, subsampled, labelled rows are concatenated across folders and written to a single Zstandard-compressed columnar file. Subsequent runs skip

ingestion and load from this cache directly. Regeneration is forced by deleting the cache file. This is documented inline so future runs do not silently use a stale cache after sampling logic changes.

### 3.2.4 Cleaning

The cleaning phase performs three operations on the materialised frame. First, null-label rows are dropped (defensive only; labels come from folder names, so this should be a no-op). Second, infinities are replaced with nulls; the actual median imputation is deferred to after the train/validation/test split so the medians are computed on the training partition only. Third, a $\log(1+x)$ transform is applied to the heavy-tailed continuous features (rates, total bytes, inter-arrival times). Binary protocol and flag indicators are excluded from the log transform.

### 3.2.5 Data Split

All experiments in this thesis use a single split type, the stratified random split. The subsampled rows are partitioned into 70/15/15 train/validation/test sets by two stratified splits in series, stratified by the fine-grained label so that every one of the 34 classes appears in all three partitions. This random split follows the convention used by most published CICIoT2023 studies, which keeps the reported numbers comparable to that body of work.

A random row split is known to be optimistic. Because each attack folder is a contiguous capture of a single session, two flows recorded moments apart can land on opposite sides of the train/test boundary, so some test rows have near-duplicate neighbours in training and the measured performance is an upper bound on what the model would achieve on genuinely unseen traffic. Two leakage-aware alternatives, a group-wise split that keeps each capture whole and a temporal split that trains on earlier captures and tests on later ones, would give a more conservative estimate; neither is used here, and both are identified as future work in Chapter 7.

### 3.2.6 Train-Only Preprocessing Fit

For the split, column medians are computed on the training rows only, nulls in the train, validation, and test partitions are imputed using those medians, and the robust scaler is fitted on training rows alone before being applied to all three partitions. The robust scaler standardises each feature by its median and interquartile range rather than its mean and standard deviation:

$$x' = \frac{x - \mathrm{median}(x)}{\mathrm{IQR}(x)}, \qquad \mathrm{IQR}(x) = Q_3(x) - Q_1(x), \tag{3.1}$$

where $Q_1$ and $Q_3$ are the first and third quartiles estimated on the training rows. Centring on the median and dividing by the IQR makes the transform insensitive to the heavy upper tails of the CIC rate and byte-count features, which would otherwise dominate a mean/variance scaler. The contract that medians, quartiles, and the scaler are fitted strictly on training rows rules out preprocessing leakage from the validation and test partitions into the fit.

## 3.3 Model Architecture and Training

### 3.3.1 MLP Topology

The model is a feed-forward multi-layer perceptron whose hidden layers each apply a linear transform followed by batch normalisation, a non-linear activation, and dropout, ending in a linear projection to the class scores. The number and width of the hidden layers, the dropout rate, and the activation are not fixed by hand: they are selected per task by the hyperparameter search of Section 3.3.4, with the resulting configurations listed in Table 3.2. Both selected networks take the 25-feature input vector and apply batch normalisation and dropout after every hidden layer:

$$\begin{aligned} \text{binary (2-class):} \quad & 25 \to 512 \to 192 \to 512 \to 2 \\ \text{8-class:} \quad & 25 \to 352 \to 256 \to 320 \to 64 \to 8 \end{aligned}$$

where each arrow other than the final one denotes the Linear–BatchNorm–activation–Dropout block above.

Batch normalisation [13], applied after each hidden linear layer, standardises that layer's pre-activations across the mini-batch to zero mean and unit variance before rescaling them by two learned parameters $(\gamma, \beta)$:

$$\mathrm{BN}(z) = \gamma \, \frac{z - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} + \beta, \tag{3.2}$$

where $\mu_{\mathcal{B}}$ and $\sigma_{\mathcal{B}}^2$ are the per-feature mean and variance over the current batch $\mathcal{B}$ and $\epsilon$ is a small constant for numerical stability.

Two design elements are held fixed across both networks regardless of the searched depth and width. Batch normalisation (Equation 3.2) after every hidden layer stabilises training under the large class-weight range: at the 8-class granularity the heaviest class weight is several times the lightest, and gradients are noisier without it. Dropout after every hidden layer provides regularisation; the search settled on moderate rates (around 0.10–0.13, Table 3.2) rather than the more aggressive 0.5 often

used on smaller datasets, which is consistent with the cleaned training set already holding several million rows.

### 3.3.2 Class-Weighted Cross-Entropy

Class weights are computed using the standard balanced-frequency rule applied to the encoded training labels, then passed to the cross-entropy loss. For a problem with $K$ classes, $N$ training samples, and $n_c$ samples in class $c$, the weight of class $c$ is

$$w_c = \frac{N}{K\, n_c}, \tag{3.3}$$

so that rare classes receive proportionally larger weight (and $\sum_c n_c w_c = N$). These weights enter the multi-class cross-entropy loss applied to the softmax outputs $\hat{y}_{i,k}$:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \sum_{k=1}^{K} w_k \, y_{i,k} \log\big(\hat{y}_{i,k}\big), \tag{3.4}$$

where $y_{i,k}$ is the one-hot ground-truth indicator for sample $i$ and class $k$. Equation (3.4) is the multi-class, class-weighted generalisation of the binary cross-entropy in Equation (2.3). Weighting is preferred over resampling because SMOTE would generate synthetic flow vectors that have no operational meaning (a half-and-half SYN-flood / DNS-spoofing flow does not correspond to any real attack), and weighted batch sampling would be redundant with weighted cross-entropy.

### 3.3.3 Optimiser, Schedule, and Early Stopping

Training uses the AdamW optimiser for both tasks, with the initial learning rate and the batch size selected per task by the search (Table 3.2). A learning-rate schedule halves the rate after two consecutive epochs without validation-loss improvement, and early stopping with a patience of five epochs caps training at a maximum of fifty epochs. The batch sizes used (2048 for the binary task and 4096 for the 8-class task) are large enough to amortise the batch-normalisation overhead while keeping the per-step gradient meaningful.

The training loop tracks training loss, validation loss, and the current learning rate per epoch, restoring the best validation-loss state at the end. The best state is what is serialised; the final-epoch state is discarded.

### 3.3.4 Hyperparameter Selection

The architecture and optimisation settings above are not arbitrary: they were selected by an automated hyperparameter search using the Optuna framework [1] rather than fixed by hand. A Tree-structured Parzen Estimator (TPE) sampler explores the search space summarised in Table 3.1 over thirty trials, each trained for ten epochs on a 100,000-row stratified subsample of the training set. The objective maximised is the validation macro-F1, chosen over validation loss so the search rewards balanced performance across the imbalanced classes rather than overall likelihood. The search is run separately for each task, so the selected configurations differ by task; they are listed in Table 3.2. Each selected configuration is then retrained to convergence (up to fifty epochs with early stopping) on the full training set and used for all reported results. The Random Forest baseline is tuned by the same Optuna TPE procedure over thirty trials, also maximising validation macro-F1, and its selected hyperparameters appear alongside the MLP in Table 3.2.

| **Hyperparameter** | **Search range** |
|---|---|
| Hidden layers | 2–4 |
| Units per layer | 64–512 (step 32) |
| Dropout | 0.1–0.5 |
| Activation | ReLU, ELU, LeakyReLU |
| Learning rate | $10^{-4}$–$10^{-2}$ (log scale) |
| Optimiser | Adam, AdamW |
| Batch size | 2048, 4096, 8192 |

Table 3.1: Optuna TPE search space for the MLP. Thirty trials, ten epochs each on a 100,000-row subsample, validation macro-F1 as the objective.

## 3.4 Evaluation Methodology

### 3.4.1 Classification Metrics

Every (granularity, model) combination is evaluated under the same protocol: accuracy, precision, recall, and F1, reported both per class and as macro and support-weighted averages, alongside the row-normalised confusion matrix. The four base metrics derive from the per-class one-versus-rest counts: true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN). Accuracy is the fraction of flows classified correctly; precision is the share of flows predicted as a class that truly belong to it; recall is the share of a class's flows that are recovered; and F1 is the

| Hyperparameter | Binary (2-class) | 8-class |
|---|---|---|
| **MLP** | | |
| Hidden layer widths | 512, 192, 512 | 352, 256, 320, 64 |
| Dropout | 0.13 | 0.10 |
| Activation | ReLU | LeakyReLU |
| Optimiser | AdamW | AdamW |
| Learning rate | $1.6 \times 10^{-3}$ | $9.5 \times 10^{-3}$ |
| Batch size | 2048 | 4096 |
| **Random Forest** | | |
| Trees ($n_{\text{estimators}}$) | 300 | 450 |
| Max depth | 25 | 25 |
| Min samples split | 7 | 6 |
| Min samples leaf | 3 | 4 |
| Max features | 0.7 | 0.7 |
| Class weight | balanced | balanced |

Table 3.2: Hyperparameters selected by the Optuna TPE search for each task, used for all reported results. The Random Forest `max_depth` is capped at 25 to keep the serialised forest tractable.

harmonic mean of precision and recall, which stays low unless both are high:

$$\begin{aligned} \text{Accuracy} &= \frac{TP+TN}{TP+TN+FP+FN}, & \text{Precision} &= \frac{TP}{TP+FP}, \\ \text{Recall} &= \frac{TP}{TP+FN}, & \text{F1} &= \frac{2\,\text{Precision}\cdot\text{Recall}}{\text{Precision}+\text{Recall}}. \end{aligned} \tag{3.5}$$

For the multi-class tasks each score is averaged over classes two ways, because they answer different questions under class imbalance. The *weighted* average weights each class's score by its support, so it reflects the traffic mix as it actually occurs but can be dominated by the large DDoS and Mirai classes. The *macro* average weights every class equally, so it exposes weakness on the rare classes that the weighted average can hide. Aggregate numbers are always accompanied by the per-class report.

### 3.4.2 Confidence Calibration

Each verdict is shown with a confidence score, the largest of the model's softmax probabilities. For that number to be useful it should mean what it says: among the flows a model calls attacks with 90% confidence, about 90% should truly be attacks. Modern neural networks rarely satisfy this out of the box; they tend to be *overconfident*, reporting peak probabilities well above their actual accuracy [12]. The reason lies in the training objective. Cross-entropy is minimised by driving the correct-class probability towards one, and once a flow is already classified correctly

the only way to reduce its loss further is to widen the gap between the correct logit and the rest. Training therefore keeps inflating logit magnitudes long after the predictions have stopped changing, and the softmax peak saturates towards one. Class-weighted training sharpens this on the classes it emphasises, which is why (Section 4.8) the binary model, dominated by a single heavily-weighted attack class, ends up markedly more overconfident than the eight-class model.

*Temperature scaling* corrects the scores without touching the predictions. A single scalar $T > 0$ is fitted on the validation-set logits $\mathbf{z}$ by minimising the negative log-likelihood, and the served probabilities are recomputed as $\mathrm{softmax}(\mathbf{z}/T)$; a value $T > 1$ flattens the distribution and pulls the inflated peak back down towards the empirical accuracy. The property that makes this safe is monotonicity: dividing every logit by the same positive constant leaves their ordering intact, so the largest logit stays the largest. The predicted label is therefore unchanged, and with it the accuracy, precision, recall, and the entire confusion matrix; only the reported probability moves. Calibration is in this sense a free correction, improving the honesty of the confidence number at no cost to any classification metric. It also explains why a single parameter is preferred over the richer per-class scalings, which have enough freedom to overfit the validation set and perturb the predictions [12].

Calibration quality is measured by the Expected Calibration Error (ECE): predictions are sorted into fifteen equal-width confidence bins, and ECE is the support-weighted average gap between the mean confidence and the mean accuracy within each bin, so a perfectly calibrated model scores zero. The fitted temperatures and the resulting change in ECE are reported in Section 4.8. The distinction is not cosmetic for an alerting tool: an operator who triages by confidence, or suppresses alerts below a threshold, is trusting that a high score really does mean a high chance of attack, and an uncalibrated 99% that is right only four times in five would quietly undermine that decision.

### 3.4.3 Latency Benchmark

For each trained model, the benchmark loop runs $1{,}000$ inference passes per batch size in $\{1, 32, 256, 1024\}$ on CPU after a warmup of one hundred passes, with the timer wrapped around the feature scaling step *and* the model forward pass. The pipeline reports the p50, p95, and p99 of the per-batch latency in milliseconds and the implied flows-per-second throughput. The acceptance criterion, later formalised as NFR-1 (Section 5.3), is a sub-100 ms p95 end-to-end latency on a consumer laptop CPU at any of those batch sizes.

# Chapter 4

# Results and Discussion

This chapter reports the experimental results of the methodology presented in Chapter 3: data ingestion statistics, exploratory data analysis findings that shaped the feature set, model training dynamics, and classification performance across the two evaluated granularities (binary and 8-class attack-family) on the random split. It closes with a cross-model comparison and a discussion of the observed generalisation behaviour. The performance targets referenced here (NFR-2 and NFR-5) are formalised in the requirements of the application, in Section 5.3.

## 4.1 Data Ingestion and Class Distribution

The ingestion pipeline processed all 34 CICIoT2023 source folders (the 33 attack types plus benign) and applied deduplication followed by per-class subsampling at a cap of 200,000 rows per fine-grained class. Figure 4.1 summarises the row counts at each stage.

The waterfall (Figure 4.1) reduces the 46,776,700 raw records to a 4,448,202-row working set in two stages, and two features of this reduction matter. First, the duplicate fraction is unusually high: 53.7% of the raw records are byte-identical to another row in the same folder. This is a known property of the dataset and has been reported by other authors [21]. Deduplicating before sampling is therefore critical: without this step, each sampled row would carry implicit near-duplicate neighbours, inflating test accuracy to an optimistic and meaningless level. Second, the 200,000-row-per-class cap then discards 79.4% of the 21,636,191 unique rows, but it only limits the large classes. After deduplication the majority class (DDoS-ICMP_Flood) still has over 200,000 unique rows, so the cap reduces it to 200,000, whereas the rarest web-based attacks (XSS, Backdoor_Malware, CommandInjection) have fewer than 5,000 unique rows each, far below the cap, so they are kept in full. The cap therefore narrows the imbalance between the largest and smallest classes but does not remove

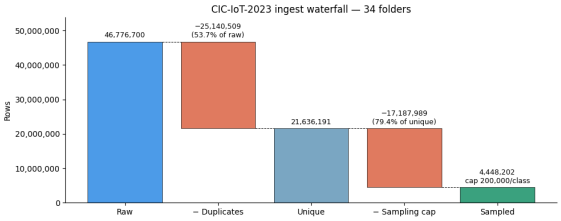


Figure 4.1: CICIoT2023 data reduction waterfall: raw records, exact duplicates removed, per-class cap applied, and the resulting working set.

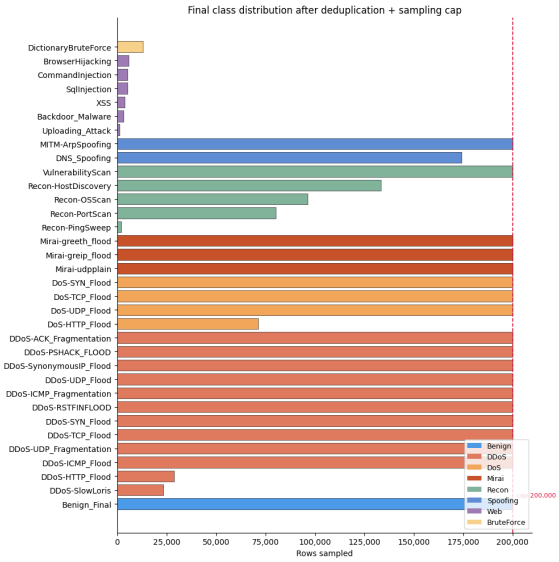


Figure 4.2: Final class distribution across the 34 fine-grained labels after deduplication and sampling; the dashed line marks the per-class cap.

it.

Figure 4.2 shows the class distribution after deduplication and sampling at the fine-grained 34-class level. Grouped into the eight attack families, the same working set is dominated by DDoS at 46.1%, followed by DoS (15.1%), Mirai (13.5%), and Reconnaissance (11.5%), with Benign at only 4.5% and the web-based family below 1%. The volumetric families sit at the cap while the rare web-based attacks fall far below it, which reflects the original dataset imbalance rather than the sampling cap.

## 4.2 Exploratory Data Analysis

Exploratory analysis informed two feature-set decisions: the removal of near-zero-variance binary features and the logging transform applied to heavy-tailed continuous features. The binary-flag variances were computed on the full cleaned dataset, while the continuous-feature correlations and the heavy-tail diagnostics used a 50,000-row random subsample; both were carried out before the train/validation/test split.

### 4.2.1 Continuous Feature Correlations

Figure 4.3 shows the Pearson correlation matrix of the 18 continuous features after the $\log(1 + x)$ transform.

Two perfectly collinear pairs are clearly visible: `AVG` packet length and `Tot_size` (total bytes, which is average $\times$ count), and `Std` (standard deviation of packet lengths) and `Variance` (its square). From each pair the redundant member (`Tot_size` and

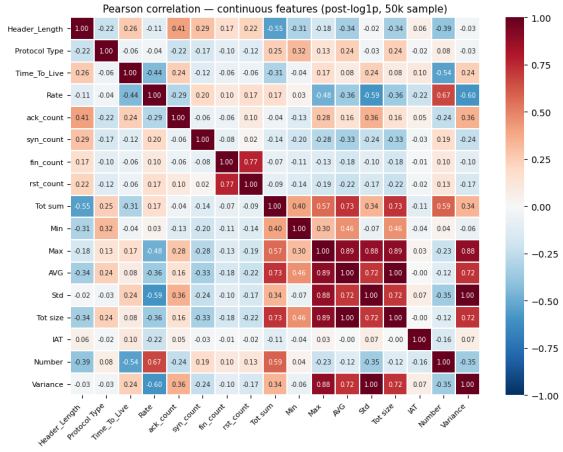


Figure 4.3: Pearson correlation matrix of the 18 continuous CICIoT2023 features on a 50,000-row post-log1p sample.

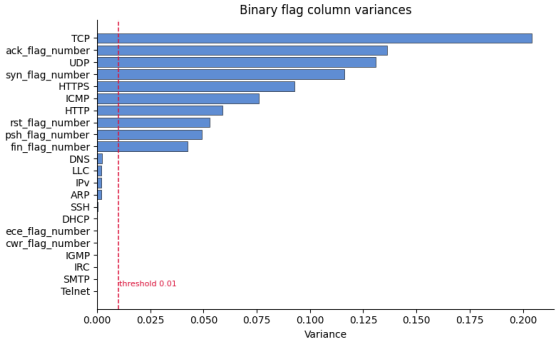


Figure 4.4: Empirical variance of the 21 binary flag and protocol-indicator features, in descending order; the dashed red line marks the 0.01 threshold.

`Variance`) is dropped, keeping `AVG` and `Std`: the discarded column carries no information its partner does not, so a tree ensemble would select the same split point on either feature and the MLP is unaffected. The `IAT` (inter-arrival time) feature is nearly orthogonal to all others, which is consistent with its role as a timing feature rather than a size-based statistic.

### 4.2.2 Binary Flag Feature Variance

The 21 binary protocol-indicator and TCP flag columns vary widely in empirical variance. Figure 4.4 shows these variances in descending order.

Features with variance below 0.01 are close-to-constant across the training sample and contribute noise rather than signal: a near-constant feature effectively tells the model nothing about class membership. Twelve such features are identified and removed. Together with the two redundant continuous columns dropped above, this prunes the set to the 25 features that form the input vector used by all models, ensuring that the scaler and the MLP are not exposed to the collinear or near-zero-variance columns that would otherwise require special handling.

## 4.3 Feature Importance

Understanding *which* features drive a model's decisions is a recurring concern in the explainable-AI-for-IDS literature, where post-hoc attribution is used to make deep detectors auditable rather than opaque (Section 2.2.2) [2]. To understand which of the 25 retained features carry the most discriminative weight, *global* permutation importance (the drop in model performance when a single feature's values are

randomly shuffled, breaking its association with the label) was computed on the Random Forest baseline. Figure 4.5 shows the top 20 features by this importance score.

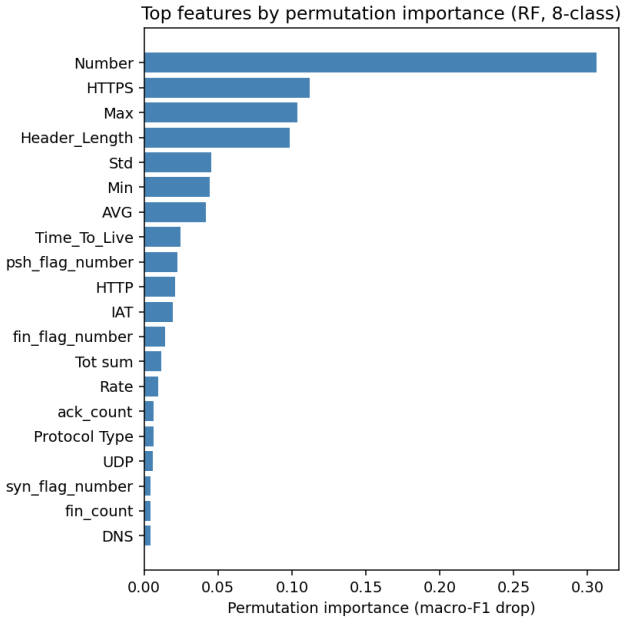


Figure 4.5: Top 20 features by permutation importance (drop in macro-F1 under feature shuffling) for the Random Forest baseline (8-class, random split).

The single most important feature by a wide margin is `Number`, the packet count within the flow window: scripted attack traffic produces highly regular packet counts that differ sharply from the variable counts of benign flows, making it the strongest single discriminator across attack families. The `HTTPS` and `HTTP` protocol indicators rank next, separating web-oriented traffic from raw-socket floods, while `Header_Length` distinguishes protocol families at the coarsest level. The packet-length statistics (`Min`, `Max`, `Std`) and `Time_To_Live` contribute in the mid-range, capturing the size and network-hop signatures that help separate volumetric floods, and the TCP flag features (`psh_flag_number`, `syn_flag_number`) add handshake-pattern information. Because permutation importance here measures the drop in *macro*-F1, it rewards features that help the minority application-layer classes disproportionately, which is why the broadly-informative count and protocol-indicator features outrank individual flag counts. The long low-importance tail in Figure 4.5 confirms that the pruned 25-feature set carries little redundant signal.

## 4.4 Model Training Dynamics

Figure 4.6 shows the training and validation loss curves for the MLP on the random split at both the binary and 8-class granularities.


Figure 4.6: MLP training and validation loss (class-weighted cross-entropy) on the random split. Left: binary (2-class) granularity. Right: 8-class granularity.

The binary model reaches its best validation loss around epoch 7 and early-stops at epoch 12, and the 8-class model peaks around epoch 4 and stops at epoch 9; in both cases early stopping triggers at patience 5 after the best validation checkpoint. The loss curves show two further effects worth noting. In the binary case, the validation loss oscillates in the early epochs before settling into a stable descent, reflecting the tension between the heavily weighted minority-class gradient signals and the majority-class (DDoS) signal. In the 8-class case the validation loss sits below the training loss throughout, which is expected rather than anomalous. The training loss is computed with dropout active and batch normalisation in training mode, and is averaged over the epoch while the weights are still improving (so the epoch-1 average is inflated by the near-random early batches), whereas the validation loss is computed at the end of the epoch with dropout disabled and batch normalisation in evaluation mode. Both losses use the same class-weighted criterion, so the gap reflects this train/eval difference, not the weighting. After its epoch-4 minimum the validation loss rises while the training loss keeps falling, the onset of mild overfitting; early stopping responds by restoring the epoch-4 checkpoint, which is the model used for all reported 8-class results. The learning-rate schedule (halving on two consecutive non-improving epochs) is visible as the steeper descent segments followed by plateau phases.

## 4.5 Binary Classification Performance

Figure 4.7 shows the row-normalised confusion matrix for the MLP on the random-split binary (2-class) test set.

The binary classifier achieves high Benign recall (0.98): under 2% of legitimate flows are falsely flagged as attacks. Attack recall is 0.91, meaning 9% of malicious flows are missed. In a deployed IDS this trade-off is configurable via the classification threshold; the model's softmax output provides a continuous score for threshold tuning.

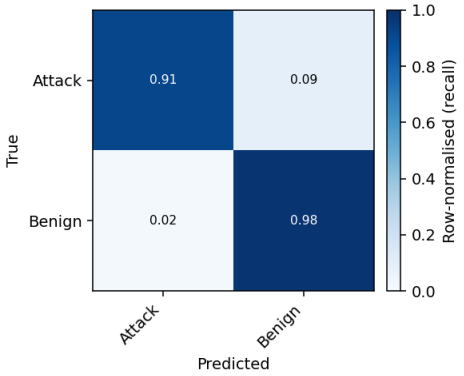


Figure 4.7: Row-normalised confusion matrix (per-class recall) for the MLP on the random-split 2-class test set.

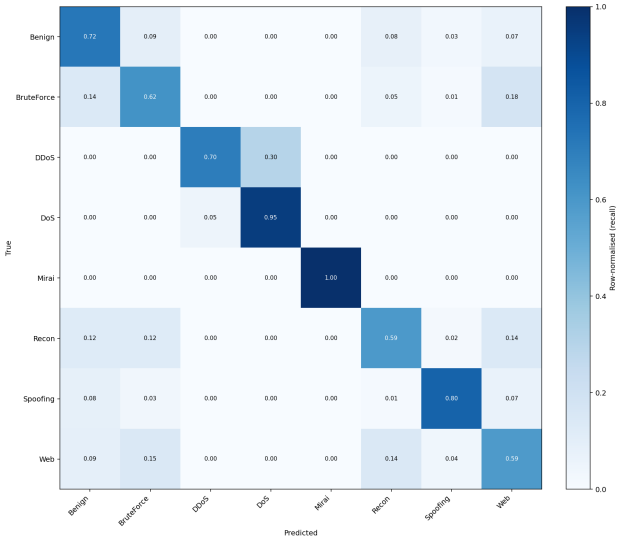


Figure 4.8: Row-normalised confusion matrix for the MLP on the random-split 8-class test set.

## 4.6 Attack-Family Classification Performance

Figure 4.8 shows the row-normalised confusion matrix for the MLP on the random-split 8-class test set.

Table 4.1 summarises the per-class recall from Figure 4.8 alongside the main confusion targets.

| Class | Recall | Main misclassification |
|---|---|---|
| Mirai | 1.00 | None (near-perfect) |
| DoS | 0.95 | 5% predicted as DDoS |
| Spoofing | 0.80 | 8% Benign, 7% Web |
| Benign | 0.72 | 9% BruteForce, 8% Recon |
| DDoS | 0.70 | 30% predicted as DoS |
| BruteForce | 0.62 | 18% Web, 14% Benign |
| Recon | 0.59 | 14% Web, 12% BruteForce |
| Web | 0.59 | 15% BruteForce, 14% Recon |

Table 4.1: Per-class recall on the random-split 8-class test set (MLP). Classes ordered by recall from highest to lowest.

The per-class recall splits into a strong and a weak group. Mirai stands apart, reaching near-perfect recall, the highest of any class, because the CICIoT2023 variants (Mirai-greith, Mirai-greip, Mirai-udpplain) all generate high-rate, fixed-payload floods over specific protocols and so produce an unusually homogeneous fingerprint that the model separates easily. The DDoS and DoS families, by contrast, are confused in both directions: both are volumetric floods differing mainly in the number of coordinated source hosts, and at the packet-feature level they share high rates, short

inter-arrival times, and dominant SYN or UDP patterns, so the 30% DDoS-to-DoS confusion (the reverse is far smaller, 5%) is unsurprising and matches published results on similar datasets [17]. An operational IDS can tolerate this confusion, since both families call for the same response (rate-limiting and upstream filtering) and neither bleeds significantly into Benign.

The weaker classes are the application-layer attacks, which cluster together rather than collapsing into Benign. Brute Force (0.62 recall), Reconnaissance (0.59), and Web (0.59) are confused mainly with one another: Web with Brute Force (15%) and Reconnaissance (14%), Reconnaissance with Web (14%) and Brute Force (12%). This happens because all three are distinguished by application-layer payload content that the header-and-metadata features do not carry, leaving the model only coarse rate and size cues to tell them apart. Spoofing, by contrast, is recovered well on this run (0.80 recall), with only minor leakage to Benign (8%) and Web (7%). In short, the features describe how packets move, not what they carry, so the families that differ mainly in payload are the ones that remain hardest to separate from each other.

## 4.7 Model Comparison

The same random split and evaluation protocol were applied to a Random Forest baseline. Figure 4.9 compares weighted F1 for both models on the binary and 8-class tasks; accuracy and macro F1 are reported in Table 4.2.

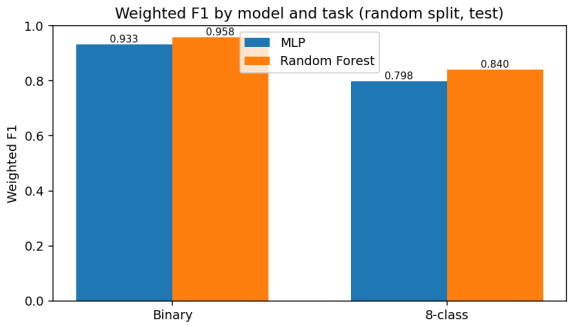


Figure 4.9: Weighted F1 for both models on the binary and 8-class tasks (random split, test set); full metrics in Table 4.2.

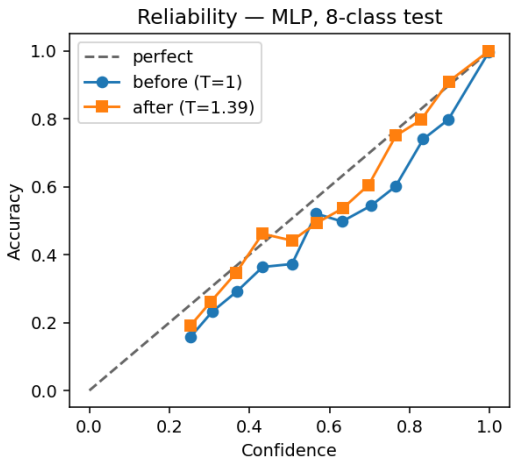


Figure 4.10: Reliability diagram for the MLP on the random-split 8-class test set, before and after temperature scaling.

Table 4.2 presents the test- and validation-set metrics for the two models.

Three results stand out from Table 4.2. First, no model reaches a weighted F1 of 0.85 at the 8-class granularity: the Random Forest tops out at ≈0.840 and the MLP at

| Model | Test Acc. | Test W-F1 | Test Macro-F1 | Val W-F1 | Val Macro-F1 |
|---|---|---|---|---|---|
| MLP | 0.775 | 0.798 | 0.631 | 0.799 | 0.632 |
| Random Forest | 0.831 | 0.840 | 0.701 | 0.840 | 0.701 |

Table 4.2: Summary of key metrics on the random split (8-class granularity), held-out test set. W-F1 = weighted F1 (per-class F1 averaged by class support); Macro-F1 = unweighted (macro-averaged) F1, weighting every class equally.

≈0.798. The 0.85 acceptance bar (NFR-2, Section 5.3) is instead met comfortably on the binary task (Section 4.5), where weighted F1 reaches ≈0.93 for the MLP and ≈0.96 for the Random Forest; the 8-class attack-family task is reported as the harder secondary objective. Its shortfall reflects what transport-layer features can and cannot express: they do not carry the application-layer signal these families differ by (Section 4.9). The model is not the limiting factor. Second, the Random Forest outperforms the MLP by about 4 percentage points on weighted F1, 6 on accuracy, and 7 on macro F1. This result is consistent with the growing body of evidence that gradient-boosted trees and random forests are competitive (and often superior) to MLPs on tabular tasks when the sample count is in the low millions [14]. The MLP remains the primary model for the demonstration system because its inference is implemented in a single portable PyTorch call, without the additional sklearn dependency; the tree baseline serves its designed role of confirming that the deep model is not strictly worse. Third, the macro F1 gap (MLP: 0.631 vs Random Forest: 0.701) is comparable to the weighted F1 gap, and the low macro F1 of both models confirms that the difficulty is concentrated in the minority classes (particularly Web, Reconnaissance, and Brute Force), while performance on the dominant DDoS and Mirai families (which drive weighted F1) is far stronger.

## 4.8 Confidence Calibration

The MLP confidences are calibrated by temperature scaling, fitting a single scalar $T$ on the validation logits and rescaling the softmax by $1/T$; the method and the Expected Calibration Error (ECE) used to measure it are defined in Section 3.4.2. This section reports the fitted temperatures and the resulting calibration on the test set.

On the 8-class task the fitted temperature is $T = 1.39$, reducing ECE from 0.056 to 0.031. On the binary task the temperature is larger, $T = 2.71$, reflecting the pronounced overconfidence that class-weighted training induces on the dominant attack class, and it lowers ECE from 0.053 to 0.031. Applying the fitted temperature in the serving path therefore yields confidence scores that more faithfully reflect

empirical accuracy, at zero cost to the predicted labels. Residual miscalibration remains and per-class (vector) scaling is left as future work.

## 4.9 Discussion

Both models generalise from train to test with small accuracy drops (below 8 percentage points). Because the split is random, these small gaps partly reflect the within-session redundancy noted in Section 3.2.5: many test rows have near-duplicate neighbours in training, so the figures reported here should be read as an upper bound rather than a deployment estimate.

The principal failure modes are the ones the design anticipated, and Section 4.6 traced each to the feature set rather than the model: the application-layer attacks (Web, Recon, Brute Force) leave no signal in a transport-layer representation, and the statistically similar DDoS and DoS floods overlap in feature space. Both are properties of the dataset's extraction methodology, not model defects.

NFR-2 (at least 0.85 weighted F1 on the *binary* test set) is satisfied by both models, which reach $\approx$0.93 (MLP) to $\approx$0.96 (Random Forest). At the 8-class granularity no model reaches 0.85 (the best attain $\approx$0.84), which the design treats as the ceiling imposed by a transport-layer feature set rather than a target failure, in keeping with the secondary status assigned to the 8-class task in NFR-2. NFR-5 (faithful reporting) is respected by reporting per-class metrics rather than only aggregate numbers, so that the minority-class weakness is visible without requiring the reader to look past a high aggregate figure, and by flagging the random split's within-session redundancy (Section 3.2.5).

# Chapter 5

# Application Requirements and Specification

This chapter defines the application that constitutes the practical contribution of the thesis: a web demonstration in which the user submits captured network traffic and receives per-flow intrusion-detection verdicts from the model of Chapter 3.

## 5.1 Project Goal and Scope

The application is a real-time intrusion detection demonstration for IoT networks, built around the model of Chapter 3. It does not train models: it consumes the artefacts produced by the offline training pipeline (the fitted scaler, the label encoder, and the trained weights) and serves them. Concretely, the application loads those artefacts, accepts either a pre-extracted feature CSV in CIC format or a raw packet capture (from which the project's own flow extractor derives the features), runs the trained model, and presents per-flow verdicts and an aggregated summary in the browser.

The system targets an interactive, single-request setting: the user supplies a sample of traffic and reviews the verdict on screen. Line-rate gateway protection (sub-millisecond per-packet inference at multi-gigabit throughput) is out of scope; this narrower sense of "real time" sets the latency thresholds below. Both classification granularities are exposed: the 2-class gate an inline deployment would use, and the 8-class attack-family view an analyst would consume.

## 5.2 Functional Requirements

The core requirement (FR-D1) is per-flow classification of submitted traffic: the demo accepts either a CSV in the CIC feature format or a raw packet capture, and

returns a per-flow verdict table with an aggregated summary. For packet captures the features are derived by the project's own extractor, which reproduces the CIC-IoT-2023 dataset's packet-windowing method so that live features match the training distribution (Section 6.3). The remaining requirements support this core. Model-variant selection (FR-D2) lists every (split, granularity) pair with trained artefacts on disk and lets the user switch without restarting. The output schema is deterministic (FR-D3): a summary line with the flow count and a per-label breakdown, followed by a table of row index, predicted label, and confidence. Input is self-validating (FR-D4): a CSV missing expected columns produces an error naming them, and a capture from which no flow can be extracted produces a readable error rather than crashing the request.

Raw traffic is never persisted (FR-D5): uploads are processed in a temporary location, removed when the request ends, and never logged; only derived result metadata (model, predicted label, confidence, mode, timestamp) may be stored for an authenticated user. The application is deployed as a permanent public service (FR-D6) at a fixed HTTPS URL with no presenter-side configuration: the backend as a Docker container on Hugging Face Spaces and the React frontend on Vercel. Because the demonstration is publicly reachable, the deployed frontend requires authentication and keeps a per-user history of past classifications (FR-D7, metadata only per FR-D5), so that each user's history stays private to them, with isolation enforced at the storage layer rather than in client code.

## 5.3 Non-Functional Requirements

End-to-end per-flow latency through the demo (CSV parsing, scaling, forward pass, result formatting) must stay below 100 ms at p95 on a consumer laptop CPU for batches of up to 1024 flows (NFR-1); packet-capture parsing is excluded because feature extraction dominates that path, and capture-to-verdict latency is reported separately. For classification performance (NFR-2), the served model must reach at least 0.85 weighted-F1 on the held-out binary test set; the harder 8-class task is a secondary objective whose weighted-F1 is expected to be lower, since the transport-layer features cannot fully separate the application-layer families.

The application must run on commodity hardware (NFR-3): CPU-only, without a GPU dependency, on both Windows and Linux, with the backend packageable as a Docker image. Operation must stay simple (NFR-4): a permanent HTTPS URL, a single command for local development, and no database, message queue, or dedicated model server, since the backend loads artefacts directly into the application process. Reporting must be faithful (NFR-5): per-class metrics always accompany aggregates so majority-class dominance cannot mask weak minority classes, and the

random split's susceptibility to within-session redundancy is stated plainly rather than hidden behind a high aggregate figure.

## 5.4 Use Cases

The application has a single actor, the *operator*: the person who opens the web interface, selects a model variant, uploads a packet capture or feature CSV, and reads the per-flow verdicts. The principal interactions are summarised in Table 5.1; the core classification path is then narrated as the representative flow.

| ID | Actor | Trigger | Outcome |
|---|---|---|---|
| UC-1 | Operator | Uploads a packet capture | Per-flow verdict table and summary; temporary capture removed afterwards. |
| UC-2 | Operator | Uploads a pre-extracted CIC CSV | Per-flow verdict table; missing columns reported as a readable error. |
| UC-3 | Operator | Selects a different model card | Result screen re-rendered under the new variant with no restart or reload. |
| UC-4 | Operator | Opens the public NEURAL IDS URL | Demo reachable from any network at a permanent URL with no presenter-side setup. |

Table 5.1: Principal use cases with their actors, triggers, and outcomes.

In the core flow (UC-1), the application stores the uploaded capture temporarily, runs the flow extractor to obtain the features, applies the trained scaler and model, renders the summary and per-flow verdict table, and removes the temporary files; if flow extraction fails, its error output is returned with an empty table. The CSV path (UC-2) is identical except that it validates the CIC column set instead of running flow extraction.

## 5.5 System Specification

This section fixes the high-level architecture and the technology choices; the detailed design follows in Chapter 6.

### 5.5.1 High-Level Architecture

The system consists of two pipelines that meet at a small set of serialised artefacts. The training pipeline runs offline and produces those artefacts. The serving pipeline is the live demo, which loads them and answers classification requests. Figure 5.1 shows the data flow.

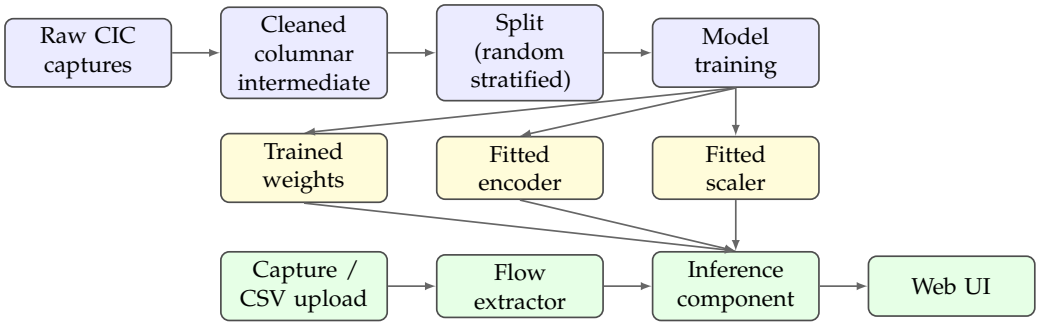


Figure 5.1: High-level architecture of the RT-IDS application. The training pipeline (blue) produces three serialised artefacts (yellow), which the serving pipeline (green) loads to answer per-flow classification requests.

### 5.5.2 Technology Stack and Rationale

The technology choices follow from the requirements. Features are scaled with the robust median/IQR scaler and labels use a reversible integer encoder; imbalance is handled by class-weighted cross-entropy (Section 3.3.2). The served model is the MLP of Section 3.3, with the Random Forest baseline trained and evaluated offline only. Packet captures are converted to features by the project's own `dpkt`-based extractor, which reproduces the windowing method the CIC-IoT-2023 dataset was itself built with; CICFlowMeter is deliberately not used, because its feature definitions differ from the dataset's (Section 6.3). The demo pairs a FastAPI backend with the NEURAL IDS React dashboard, containerised on Hugging Face Spaces and served from Vercel respectively, which yields a permanent HTTPS URL (NFR-3, NFR-4). The backend loads the artefacts directly into the application process, so no separate model server is needed.

### 5.5.3 Constraints and Assumptions

Several constraints bound the work. The 39-feature public CSV release of CICIoT2023 is the only source of training data; the 46-feature variant used by some published comparisons was not available from the official distribution, so numerical results are not directly comparable across the two feature sets (Section 2.1.4). The dataset comes from a smart-home testbed, so generalisation to industrial, medical, or vehicular networks is not claimed, and its header-and-metadata features mean detection of

application-layer web attacks is expected to be weak, a property of the feature set discussed in the results chapter. The serving target is a single-container backend with a static frontend, so distributed serving, load balancing, and high availability are out of scope. Because the demonstration is deployed as a public service, the per-user history (FR-D7) must be kept private to each user, which requires authentication; this is provided by Supabase email/password, chosen because it bundles managed authentication with a Postgres database and row-level security, so per-user isolation is enforced at the data layer without a bespoke authentication backend. Finally, the deployed model is the one trained offline; the demo does not adapt to feedback during a session.

# Chapter 6

# Application Design, Implementation and Testing

This chapter covers the application built around the machine-learning core of Chapter 3: how it is structured, how each part is implemented, and how it is tested. The data pipeline, the split, and the model itself were presented there; here the focus is the serving system that turns a trained model into something a user can upload traffic to and get a verdict from.

## 6.1 Architectural Overview

The split between the offline training pipeline and the online serving pipeline, which meet at a small set of saved artefacts, was fixed in Section 5.5; this chapter takes that split as given. The point worth adding is why the link between the two sides is kept this thin. Because the only thing passed from training to serving is a set of files, the model can be retrained with different splits, granularities, or hyperparameters without changing a line of serving code, and the serving side stays small enough to read in one sitting.

### 6.1.1 Components

The application has five parts, each with a single job so that a fault is easy to place. The **training pipeline** produces the artefacts (ingestion through to saving the model, covered in Chapter 3). The **inference component** loads one trained variant and turns feature rows into labelled predictions with confidences. The **flow extractor** turns a packet capture into the model's feature rows, reproducing the dataset's own packet-windowing method. The **web application** finds the available variants on disk, shows the user interface, sends uploads to the inference component, and formats the verdicts. Underneath them all, the **project specification** fixes the invariants – the

feature set, the split, the classification granularities, and the modelling constraints – that every part must respect.

### 6.1.2 Artefact Contract

Each trained variant is three files: the fitted feature scaler (one per split, shared across granularities because the input features are the same whatever the labels), the fitted label encoder (one per granularity, because the label set changes), and the trained model weights. The filenames encode the split and the granularity in a fixed pattern, so a variant is identified entirely by its filenames, with no hard-coded list anywhere in the serving code. Adding a newly trained variant is therefore just a matter of dropping its three files into the artefact directory and restarting; the code that picks them up at startup is described in Section 6.4.1.

## 6.2 Inference Component

The inference component is the one place where saved artefacts meet live requests. It is created with an artefact directory, a split name, and a granularity, and loads the matching three files. Given a frame of feature rows it returns the predicted labels, the per-row confidences, the full softmax probabilities, and the class names in the encoder's order.

Its preprocessing has to reproduce the training-time steps of Chapter 3 exactly, or the fitted scaler would see values it was never built for. It runs those same steps with the saved scaler's transform – never a re-fit, since the scaler is fixed once trained – and adds only the two things that matter on the serving side. The expected feature columns are checked and restored to the trained order, so a reordered upload cannot silently shift them. And any null still present after the transforms is filled with a constant rather than a recomputed median, because at inference the scaler already operates in the transformed space.

The model outputs logits. These are divided by the temperature scalar found during calibration, a softmax turns them into per-class probabilities, the largest one selects the class, and the encoder maps that index back to a readable label. The reported confidence is that largest temperature-scaled probability, so it tracks the model's real accuracy rather than the raw, over-confident softmax peak.

## 6.3 Packet-to-Feature Extraction

A packet capture has to be turned into the same 25 features the model was trained on. CICFlowMeter, the CIC team's own flow extractor, is the tool one would reach

for first, but the CIC-IoT-2023 features were *not* produced with it: the dataset authors used a custom extractor that groups packets into fixed-size windows between two hosts and mean-aggregates per-packet statistics. Running CICFlowMeter would hand the model a differently-defined feature set, so it is deliberately not used. The project instead reimplements the dataset's own method in Python, parsing both `pcap` and `pcapng` with `dpkt`: packets are grouped by unordered host pair, cut into non-overlapping windows, and each window is reduced to the 25 features in the model's column order, so that captured and live traffic land in the same distribution the model learned. The few points on which the dataset paper is silent (window stride, the exact host-pair definition, the packet-length convention) are fixed explicitly and documented in the code. If a capture yields no usable window – for instance it carries no IPv4 TCP/UDP traffic – the request returns a readable error rather than crashing, which is what surfaces under FR-D4.

## 6.4 Web Application

The serving side is two cooperating layers: a **backend** – a FastAPI endpoint running in one Python process – and a **React frontend** that calls it and provides the user interface. They are joined only by the REST contract, so the frontend can be hosted separately while the backend runs locally or on Hugging Face Spaces.

### 6.4.1 Backend: FastAPI

At startup the backend scans the artefact directory, loads every complete three-file variant it finds, and keeps them in a dictionary keyed by a readable `split / granularity` name; if it finds none, it exits with a message pointing back to the training pipeline (FR-D2). It then exposes a single REST endpoint, `POST /api/classify`, which takes the uploaded file plus the chosen model type, split, and granularity, detects whether the upload is a CSV or a packet capture, runs the prediction, and returns JSON with the top label, its confidence, the full per-class probabilities, a per-flow breakdown, and the processing time. The React frontend is its only client and renders that JSON as the result screen of Section 6.4.2.

The CSV and packet-capture uploads are handled symmetrically. The CSV path checks the columns and forwards the rows directly; the capture path first writes the file to a temporary directory and runs the extractor of Section 6.3 to obtain the feature rows. After that both paths join and go through the same inference component, so the response format is identical for either input.

### 6.4.2 Frontend: NEURAL IDS

The React frontend is branded *NEURAL IDS*. Its landing screen (Figure 6.1) introduces the system and leads into the dashboard, where the operator picks a model variant, uploads traffic, and reviews a history of the runs made during the session. Only the MLP variants are served live; the Random Forest baseline of Chapter 4 is trained and evaluated offline and is not exposed here. On upload the frontend calls `POST /api/classify` and renders the returned verdict on a result screen (Figure 6.2): a confidence gauge, a ranked bar chart of all class probabilities, the run's metadata, and the per-flow breakdown by predicted label.

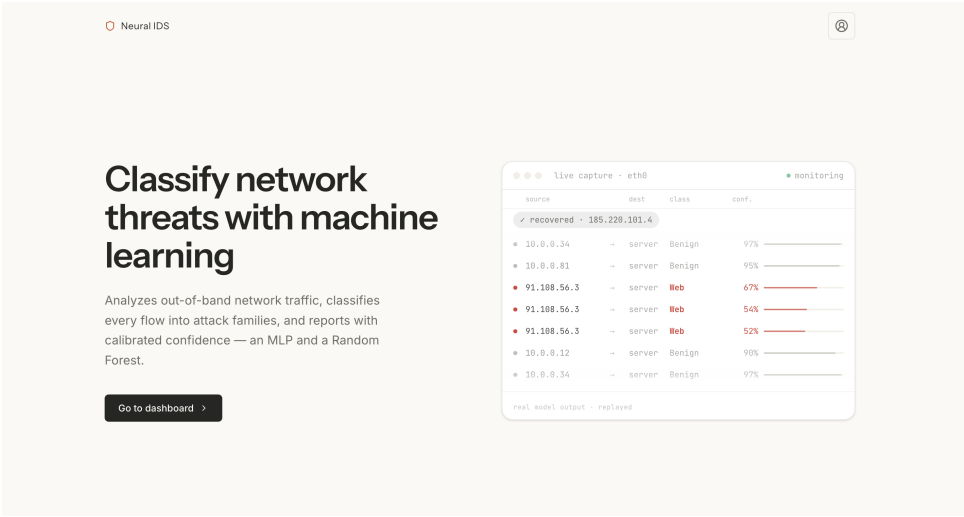


Figure 6.1: The NEURAL IDS landing screen, which introduces the system and leads into the dashboard for model selection, upload, and the session's analysis history.

### 6.4.3 Authentication, Persistence, and Deployment

Because the demo is public, each user's history has to stay private, so the frontend talks to two independent backends (Figure 6.3). The first is the *stateless inference channel*: the React app sends a cross-origin `POST /api/classify` to the FastAPI backend on Hugging Face Spaces and gets back a JSON verdict; that backend holds no database and remembers nothing between requests. The second is the *stateful identity-and-history channel*: the same app talks to a managed Supabase project for sign-in and for storing past results. The two never share server-side state, so either can be replaced without touching the other and an inference request never waits on the database.

Supabase is used because it bundles managed email/password authentication with a Postgres database and row-level security, which avoids building a separate auth service. On sign-in the client receives a JSON Web Token, attaches it to later requests, and restores the session from the stored token on reload. After each classifi-

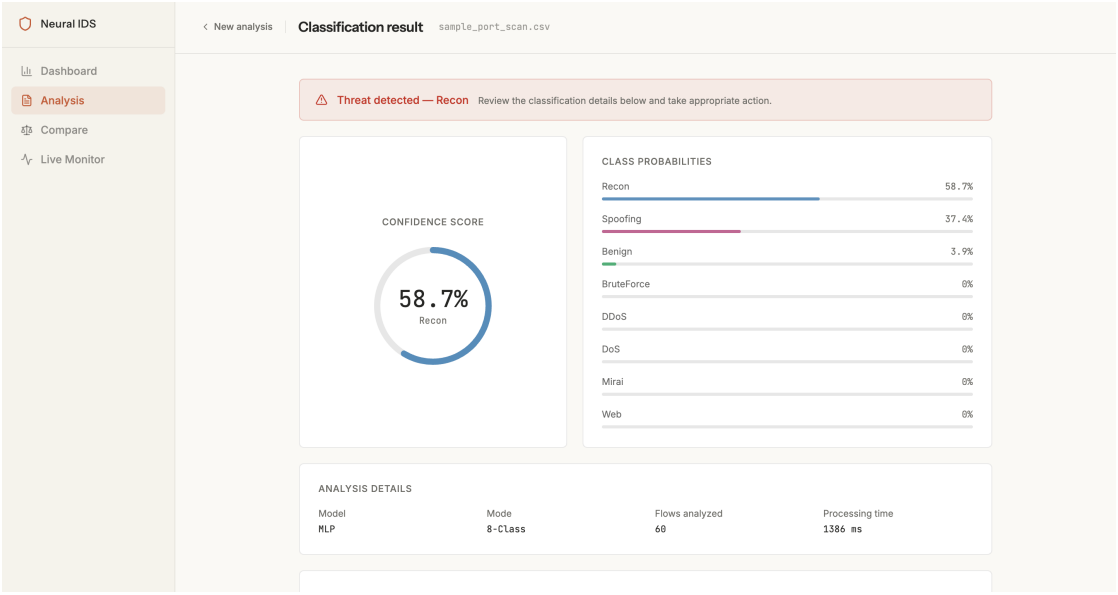


Figure 6.2: A classification result in NEURAL IDS: a port-scan capture classified at the 8-class granularity as **Recon** (reconnaissance), the correct family for scanning traffic, with Spoofing second. The screen shows the confidence gauge, the ranked class probabilities, and the run metadata. The processing time shown includes the on-demand SHAP explanation (the local feature attribution of Section 2.2.2), so it is larger than the raw inference latency of the benchmark in Section 3.4.3 (NFR-1).

cation the result screen writes one row to the `analyses` table, tagged with the user's identifier: a UUID key, the `user_id`, a server timestamp, the variant and granularity, the input type, the file name, the top label with its calibrated confidence, and the full probability vector as JSON. The uploaded traffic itself is never stored, only this derived metadata, which keeps the table consistent with FR-D5. Privacy does not depend on the client: a row-level-security policy limits every read and write to rows whose `user_id` matches the caller, so one user can never see another's history even though all rows live in one table.

For deployment the backend is packaged as a Docker image on Hugging Face Spaces and the frontend is served from Vercel, so the demo works from any browser without the presenter's laptop being reachable. For the local defence the same backend runs on the laptop over loopback, with the frontend served from the same machine. Figure 6.3 shows the result: Vercel serves the static application, which at runtime fans out to the two independent backends.

## 6.5 Implementation Notes

A few choices turn this design into working code. The offline pipeline stays within a modest memory budget by reading raw CSVs lazily and only materialising after columns are projected; the deduplicated, subsampled frame is the only large object held, and it is freed once converted to arrays. Reproducibility comes from broad-

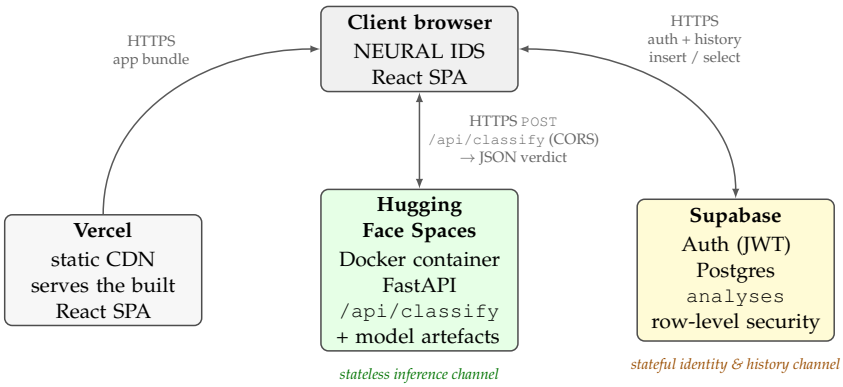


Figure 6.3: Runtime deployment topology. Vercel delivers the static React SPA (grey); at runtime the SPA drives two independent backends – a *stateless* inference service on Hugging Face Spaces (green), reached via `POST /api/classify`, and a *stateful* Supabase project (yellow) for authentication and per-user history. The two channels share no server-side state, so inference never blocks on persistence and either tier can be replaced in isolation.

casting one seed to every random source – the numerical library, the deep-learning framework on CPU and GPU, its deterministic flags, and the samplers – so re-running on the same files reproduces the metrics to within a few thousandths of an F1 point. All the constants (the per-class cap, the 25-feature selection, the active splits and granularities, the batch size, the epoch and early-stopping settings, the learning rate, and the seed) live in one configuration module shared by the notebook and the headless training command, so the two can never disagree. The runtime needs only the standard scientific-Python stack plus a lightweight web framework: there is no separate serving framework (the model is loaded straight into the application process) and no packaging beyond a single directory launched by one command, which satisfies NFR-4.

## 6.6 Testing Strategy

The application is tested at three levels: sanity checks inside the training pipeline, an artefact round-trip test per variant, and an end-to-end functional acceptance test.

### 6.6.1 Pipeline Sanity Checks

The pipeline asserts and prints checks at each step. One assertion confirms the fine-grained label map covers every attack folder exactly once; another confirms ingestion leaves no folder unmapped, so a newly added attack folder cannot be silently dropped. For each split it checks that every fine-grained label appears at least once in the training partition and warns if not. Per-folder row counts (raw, deduplicated, sampled) are printed, so a sudden change in the source data shows up

immediately.

### 6.6.2 Artefact Round-Trip Test

For every variant, after training, the pipeline reloads the freshly written files through the exact code path the demo uses and predicts on the test partition, then checks those predictions match the still-in-memory model's exactly. This caught two real bugs during development: a column-ordering drift between training and inference, and a silent dtype mismatch when arrays became tensors without an explicit precision cast.

### 6.6.3 Functional Acceptance Test

The core function is checked end-to-end on two inputs whose correct labels are known in advance, so each run either clears a fixed threshold or it does not. The benign baseline is a short capture of normal browsing made on the demo machine; it passes if at least 95% of flows are predicted Benign at the binary granularity. The flood is a SYN-flood capture against a local target made with a standard packet-crafting tool; it passes if at least 95% of flows are predicted Attack at the binary granularity and the dominant family at the 8-class granularity is a denial-of-service family. These thresholds sit well below the test-set numbers, so the test tolerates the shift between the training data and the demo machine while still failing loudly if the demo is wired to the wrong scaler or encoder.

### 6.6.4 Negative-Path Testing

Three failure cases are exercised manually, since they reproduce from a brief interactive session. A CSV missing one or more expected feature columns produces a domain error naming the missing columns (FR-D4). A non-capture file (for example a text file renamed to a capture extension) produces a flow-extraction error carrying the tool's own stderr (also FR-D4). Starting the demo with no artefacts on disk exits quickly with a message pointing back to the training pipeline (FR-D2).

## 6.7 Functional Acceptance Evidence

The acceptance test of Section 6.6.3 was run against the deployed system through the NEURAL IDS frontend, using two pre-extracted CSVs with known labels: `sample_benign_browsing.csv` (normal browsing) and `sample_syn_flood.csv` (a SYN flood against a local target). Both passed. For the SYN flood, the binary MLP labelled every flow Attack (mean calibrated confidence 99.6%) and the 8-class variant

confirmed DoS as the dominant family – both correct. For the benign capture, the binary MLP returned Benign on 98% of flows, above the 95% threshold, discharging the no-false-alarm half of the test. (Confidence here is the mean softmax probability of the dominant class after temperature scaling, not the share of flows in that class.)

Figure 6.2 shows the result screen on a third capture, a port scan: the 8-class MLP labels it Recon, the reconnaissance family, which is the expected result for scanning traffic. Together the runs show the end-to-end path working – upload, flow features, prediction, and a calibrated verdict – across benign, denial-of-service, and reconnaissance traffic.

# Chapter 7

# Conclusions and Future Work

This thesis designed, implemented, and evaluated a real-time intrusion detection system for IoT networks on the CICIoT2023 dataset, delivering a reproducible offline training pipeline and an interactive web demonstration. The pipeline removes the large exact-duplicate fraction before sampling, caps each class at 200,000 rows, and prunes the 39 public features to 25 by variance and correlation. On that input a multi-layer perceptron was trained at binary and 8-class granularities on a stratified random split, compared against a Random Forest baseline under identical metrics, and calibrated by temperature scaling. The NEURAL IDS demonstration pairs a FastAPI backend with a React dashboard that classifies uploaded captures or CSVs in under 10 ms and is deployed for remote access.

Four findings stand out. The binary weighted F1 (around 0.93 for the MLP) clears the 0.85 target of NFR-2, while the 8-class figure (around 0.80 for the MLP, 0.84 for the Random Forest) sits below it, the expected price of fine-grained family discrimination from transport-layer features alone. At the family level Mirai is near-perfectly detected (1.00 recall), DDoS and DoS are heavily confused with each other as volumetric floods, and the application-layer families (Web, Reconnaissance, Brute Force) are weakest because they are distinguished by payload content the header features do not capture, a ceiling set by the data, not the classifier. The Random Forest beats the MLP by 4 to 7 points across accuracy, weighted F1, and macro F1, consistent with tabular-learning results; the MLP is kept as the serving model because its inference is a single portable call and the gap stays within target.

The main limitations are the header-and-metadata feature set, which caps application-layer detection; the controlled 105-device testbed, which omits multi-stage campaigns and background noise; and the random split, which leaks near-duplicate flows across the train/test boundary and so reports an upper bound on deployment performance. Global temperature scaling also leaves residual per-class miscalibration (8-class ECE 0.031 after scaling).

These point to clear next steps: evaluate on leakage-aware (group-wise and

temporal) splits to quantify how much performance the random split inflates; replace the single global temperature with per-class scaling and per-class thresholds to tune the detection/false-alarm trade-off per family; extend the features with TLS-visible metadata to reach the encrypted and web-based attacks the transport-layer view currently misses; and deploy the serving component on a real IoT gateway with periodic retraining on misclassified flows to test the pipeline beyond the laboratory.

# Bibliography


[1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 2623–2631. ACM, 2019.

[2] Hassan M. J. Almohri et al. Explainable artificial intelligence for intrusion detection in iot networks: A deep learning based approach. *Expert Systems with Applications*, 238:121855, 2024.

[3] Moayad Aloqaily, Safa Otoum, Ismaeel Al Ridhawi, and Yaser Jararweh. Ai-ids: Application of deep learning to realtime web intrusion detection. In *2020 International Wireless Communications and Mobile Computing (IWCMC)*, pages 2058–2063. IEEE, 2020.

[4] Giuseppina Andresini, Feargus Pendlebury, Fabio Pierazzi, Corrado Loglisci, Annalisa Appice, and Lorenzo Cavallaro. INSOMNIA: Towards concept-drift robustness in network intrusion detection. In *Proceedings of the 14th ACM Workshop on Artificial Intelligence and Security (AISec '21)*, pages 111–122. ACM, 2021.

[5] Giovanni Apruzzese, Pavel Laskov, and Johannes Schneider. SoK: Pragmatic assessment of machine learning for network intrusion detection. In *Proceedings of the 8th IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 592–614, Delft, Netherlands, 2023. IEEE.

[6] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.

[7] Cloudflare, Inc. What is a packet? Cloudflare Learning Center, `https://www.cloudflare.com/learning/network-layer/what-is-a-packet/`. Accessed June 2026.

[8] Cloudflare, Inc. What is DDoS mitigation? Cloudflare Learning Center, `https://www.cloudflare.com/learning/ddos/ddos-mitigation/`. Accessed June 2026.

[9] Mohamed Amine Ferrag, Leandros Maglaras, Sotiris Moschoyiannis, and Helge Janicke. Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study. *Journal of Information Security and Applications*, 50:102419, 2020.

[10] Fortinet, Inc. What is an intrusion prevention system (IPS)? Fortinet CyberGlossary, `https://www.fortinet.com/resources/cyberglossary/what-is-an-ips`. Accessed June 2026.

[11] Fortinet, Inc. What is cybersecurity? Fortinet CyberGlossary, `https://www.fortinet.com/resources/cyberglossary/what-is-cybersecurity`. Accessed June 2026.

[12] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70, pages 1321–1330, Sydney, Australia, 2017. PMLR.

[13] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, volume 37, pages 448–456. PMLR, 2015.

[14] Arlind Kadra, Marius Lindauer, Frank Hutter, and Josif Grabocka. Well-tuned simple nets excel on tabular datasets. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 23928–23941, 2021.

[15] Jiyeon Kim, Jiwon Kim, Hyunjung Kim, Minsun Shim, and Eunjung Choi. CNN-based network intrusion detection against denial-of-service attacks. *Electronics*, 9(6):916, 2020.

[16] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, San Diego, CA, USA, 2015.

[17] Jan Lansky, Saqib Ali, Mokhtar Mohammadi, Mohammed Kamal Majeed, Sarkhel H. Taher Karim, Shima Rashidi, Mehdi Hosseinzadeh, and Amir Masoud Rahmani. Deep learning-based intrusion detection systems: A systematic review. *IEEE Access*, 9:101574–101599, 2021.

[18] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 4765–4774. Curran Associates, Inc., 2017.

[19] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. Kitsune: An ensemble of autoencoders for online network intrusion detection. In *Proceedings of the 25th Annual Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, USA, 2018. Internet Society.

[20] Nour Moustafa and Jill Slay. UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In *2015 Military Communications and Information Systems Conference (MilCIS)*, pages 1–6. IEEE, 2015.

[21] Euclides Carlos Pinto Neto, Sajjad Dadkhah, Raphael Ferreira, Alireza Zohourian, Rongxing Lu, and Ali A. Ghorbani. Ciciot2023: A real-time dataset and benchmark for large-scale attacks in iot environment. *Sensors*, 23(13):5941, 2023.

[22] Martin Roesch. Snort – lightweight intrusion detection for networks. In *Proceedings of the 13th USENIX Conference on System Administration (LISA '99)*, pages 229–238, Seattle, Washington, USA, 1999.

[23] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116, 2018.

[24] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.

[25] Mahbod Tavallaee, Ebrahim Bagheri, Wei Lu, and Ali A. Ghorbani. A detailed analysis of the KDD CUP 99 data set. In *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, pages 1–6. IEEE, 2009.

[26] R. Vinayakumar, Mamoun Alazab, K. P. Soman, Prabaharan Poornachandran, Ameer Al-Nemrat, and Sitalakshmi Venkatraman. Deep learning approach for intelligent intrusion detection system. *IEEE Access*, 7:41525–41550, 2019.

[27] R. Vinayakumar, K. P. Soman, and Prabaharan Poornachandran. Applying convolutional neural network for network intrusion detection. In *2017 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*, pages 1222–1228. IEEE, 2017. Frequently cited under year 2018 due to indexing date.

[28] Zihan Wu, Hong Zhang, Penghai Wang, and Zhibo Sun. RTIDS: A robust transformer-based approach for intrusion detection system. *IEEE Access*, 10:64375–64387, 2022.